

2023 - 2024



UNIVERSITY OF
PATRAS
ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

Ψηφιακή επεξεργασία και ανάλυση εικόνας
Εργαστηριακή άσκηση Μέρος Α

ΜΠΑΡΑΚΟΥ ΓΕΩΡΓΙΑ

ΑΜ : 1090075

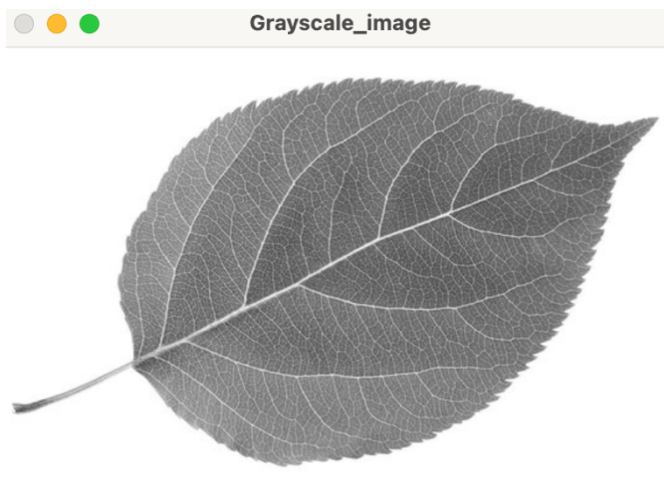
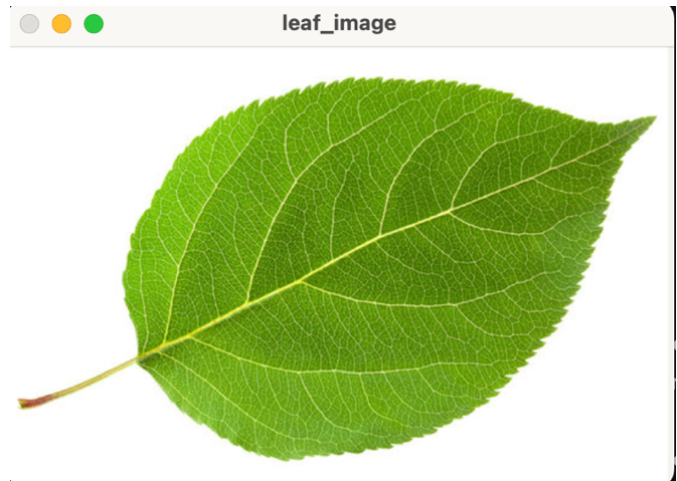
4ο έτος σπουδών

Περιεχόμενα

<i>Ερώτημα 1.1 : Περιγραφή σχήματος</i>	<i>2</i>
<i>Ερώτημα 1.2 : Συμπίεση εικόνας.....</i>	<i>7</i>
<i>Ερώτημα 1.3 : Βελτίωση εικόνας – Φιλτράρισμα θορύβου.....</i>	<i>10</i>
<i>Ερώτημα 1.4 : Βελτίωση εικόνας – Εξίσωση ιστογράμματος</i>	<i>13</i>
<i>Ερώτημα 1.5 : Αποκατάσταση εικόνας (Wiener deconvolution).....</i>	<i>16</i>
<i>Ερώτημα 1.6 : Ανίχνευση ακμών</i>	<i>18</i>
<i>Ερώτημα 1.7 : Κατάτμηση εικόνας</i>	<i>21</i>
<i>Κώδικας.....</i>	<i>24</i>

Ερώτημα 1.1 : Περιγραφή σχήματος

- 1) Αρχικά, παραθέτουμε την αρχική εικόνα που μας δόθηκε :

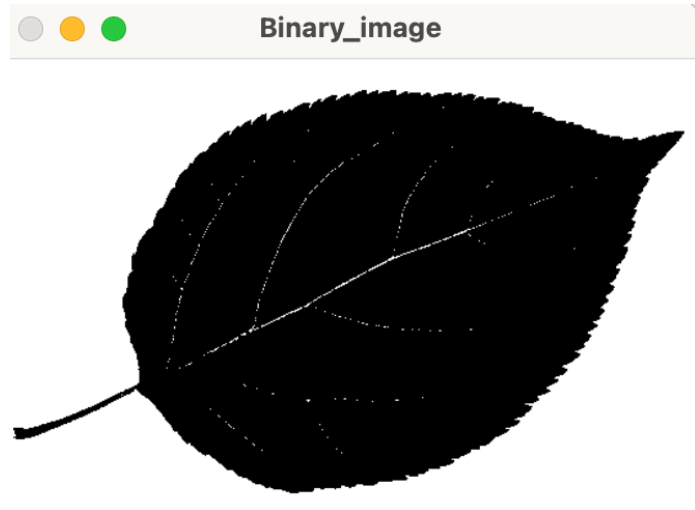


Ζητήθηκε η μετατροπή της αρχικής εικόνας σε grayscale image. Αυτό σημαίνει ότι μετατρέψαμε τις τιμές των pixel από RGB σε αποχρώσεις του γκρι με την εντολή :

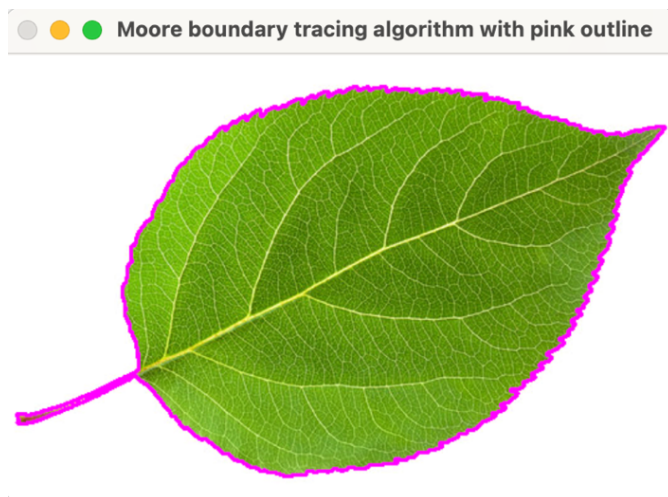
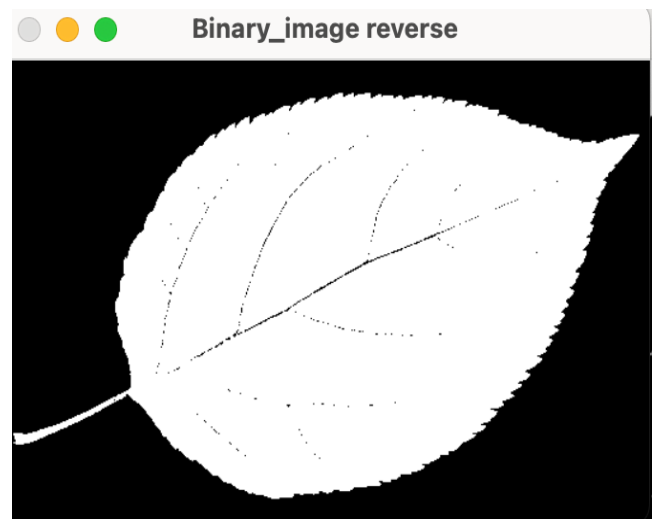
```
grayscale_image  
=cv2.cvtColor(leaf_image,  
cv2.COLOR_BGR2GRAY)
```

Έπειτα, κατωφλιώσαμε την grayscale εικόνα με τιμή κατωφλίου $\tau = 220$, όπου όταν η τιμή του pixel είναι μικρότερη του τ τότε γίνεται μαύρο και όταν είναι μεγαλύτερη γίνεται άσπρο :

```
_, binary_image =  
cv2.threshold(gray_image,  
220, 255, cv2.THRESH_BINARY)  
και προέκυψε η binary εικόνα :
```



- 2) Εφαρμόσαμε το αλγόριθμο Moore boundary tracing algorithm στην binary εικόνα. Αρχικά θα αντιστρέψουμε την δυαδική εικόνα έτσι ώστε το φύλλο να γίνει λευκό και το φόντο να γίνει μαύρο. Έτσι ο αλγόριθμος θα “ακολουθήσει” το περίγραμμα του λευκού αντικειμένου, στην περίπτωση του φύλλου. Τέλος επιλέγουμε το χρώμα του περιγράμματος και εισάγουμε το περίγραμμα της binary εικόνας πάνω στην αρχική:

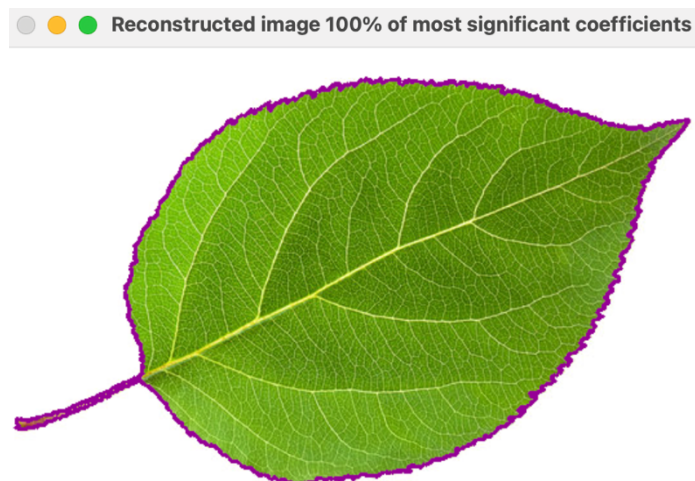


- 3) Στην συνέχεια ζητήθηκε ο υπολογισμός των περιγραφών Fourier του περιγράμματος. Αρχικά από την εικόνα με το ροζ περίγραμμα έγινε μετατροπή των σημείων που περιέχουν το περίγραμμα σε αριθμούς που τους εισήγαμε σε έναν πίνακα. Οι αριθμοί αυτοί αποτελούνται από πραγματικό (x) και φανταστικό (y) μέρος. Υπολογίσαμε τον μετασχηματισμό fourier του πίνακα :

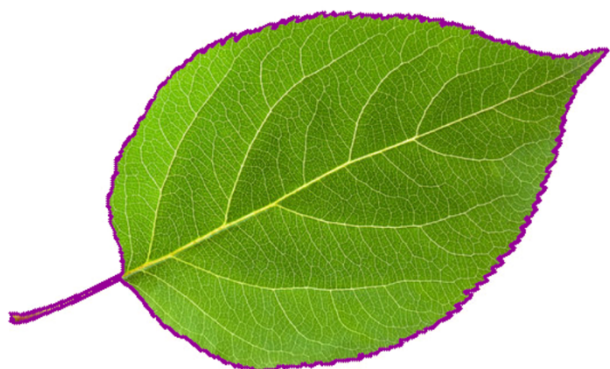
```
Περιγραφείς Fourier του ροζ περιγράμματος:  
[ 4.07478000e+05+264179.j      4.23677153e+04 +51374.76742744j  
  6.58652939e+03 -9305.16502183j ... -2.11197519e+02 -1279.32231669j  
 -4.96995197e+03 -1539.98074199j -2.84519429e+04-269216.83860392j]
```

- 4) Από τους περιγραφείς Fourier ανακατασκευάσαμε το περίγραμμα του φύλλου. Με την χρήση του IFFT θα κάνουμε την ανακατασκευή. Σε αυτή την περίπτωση ζητήθηκε να χρησιμοποιήσουμε τα παρακάτω ποσοστά των πιο σημαντικών συντελεστών. : 100, 50, 10, 1. Όσο μεγαλύτερο είναι το ποσοστό, δηλαδή όσο περισσότερους συντελεστές χρησιμοποιήσουμε για την ανακατασκευή, τόσο πιο “καλό”, λεπτομερές θα βγει το περίγραμμα. Ακόμη, για την χρήση των πιο σημαντικών συντελεστών κρατάμε τους αρχικούς και τελικούς συντελεστές, έτσι ώστε να κρατήσουμε μια πιο γενική και μία πιο λεπτομερείς πληροφορία καθώς όσο αυξάνονται οι συχνότητες τόσο περισσότερη λεπτομέρεια περιέχει.

Χρησιμοποιώντας το **ποσοστό των 100**, είναι προφανές πως θα ανακατασκευάσουμε ένα περίγραμμα ίδιο με το αρχικό, αφού χρησιμοποιούμε το 100% των συντελεστών.



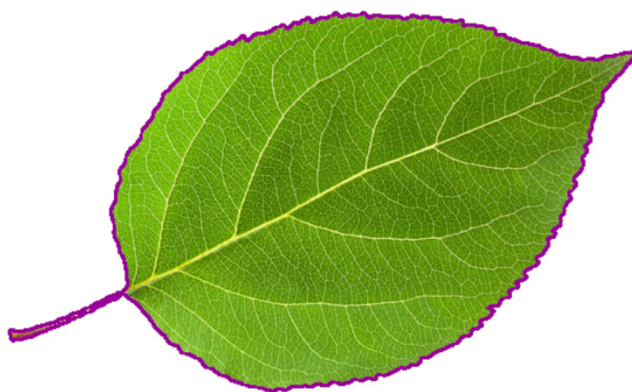
Reconstructed image 50% of most significant coefficients



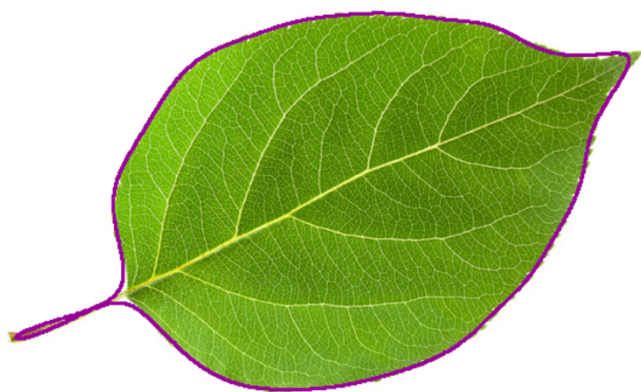
Χρησιμοποιώντας **το ποσοστό των 50**, παρατηρείται μία ελάχιστη απώλεια πληροφορίας, όχι τόσο διακριτή στο μάτι.

Έπειτα έχουμε **το ποσοστό των 10**, όπου και εδώ έχουμε λιγότερη πληροφορία.

Reconstructed image 10% of most significant coefficients



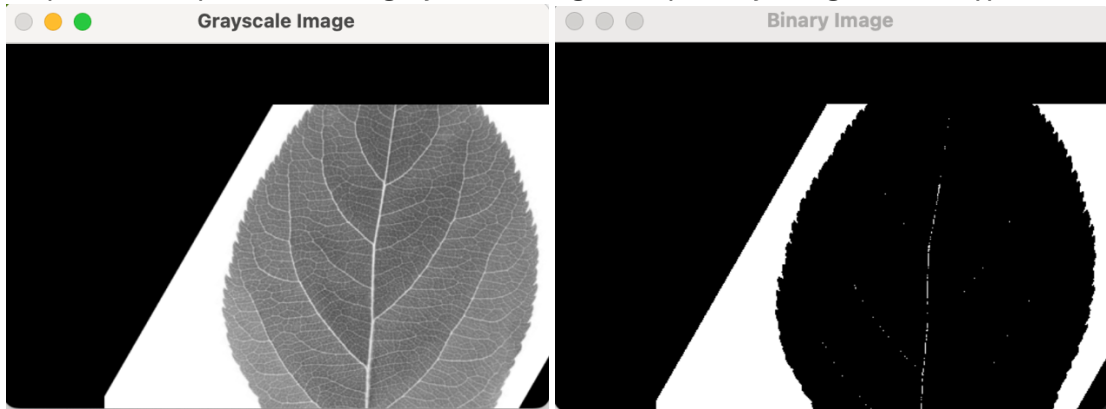
Reconstructed image 1% of most significant coefficients



Τέλος, με **το ποσοστό του 1**, είναι αισθητή η απώλεια της πληροφορίας σε σχέση με την αρχική εικόνα.

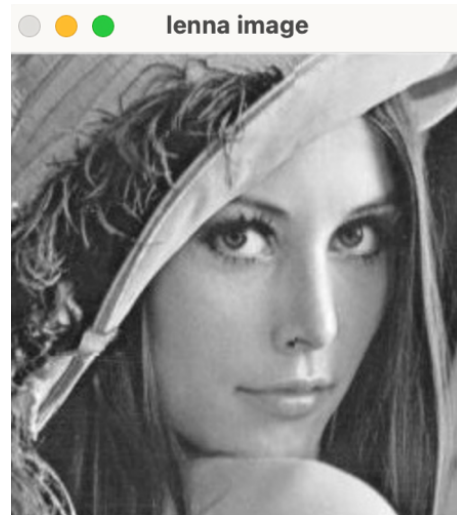
- 5) Ζητήθηκε να κάνουμε περιστροφή κατά 60 μοίρες και ολίσθηση κατά (100,60) pixels στην αρχική εικόνα με το φύλλο και να επαναλάβουμε τις παραπάνω διαδικασίες. Αρχικά, περιστρέψαμε κατά 60 μοίρες ως προς το κέντρο της και έπειτα με βάση τον άξονα του x και τον άξονα του y κάναμε την ολίσθηση αντίστοιχα για $x = 100$ και για $y = 60$.

Παρακάτω παρατίθενται οι grayscale image και η binary image αντίστοιχα.



Ερώτημα 1.2 : Συμπίεση εικόνας

- 1) Με την εφαρμογή του μετασχηματισμού 2D-DCT πραγματοποιήθηκε συμπίεση στην παρακάτω εικόνα.



Αρχικά, “σπάσαμε” την εικόνα σε block διαστάσεων 32 x 32 και σε κάθε block εκτελέσαμε τον 2D-DCT. Με την συμπίεση αφαιρέθηκαν οι πιο υψηλές συχνότητες, αφού είναι αυτές που περιέχουν την περισσότερη πληροφορία στην εικόνα. Εκτυπώσαμε τους συντελεστές που προέκυψαν από τον μετασχηματισμό και εμφανίσαμε κάθε block μετά την εφαρμογή του 2D-DCT σε μία ενιαία εικόνα.

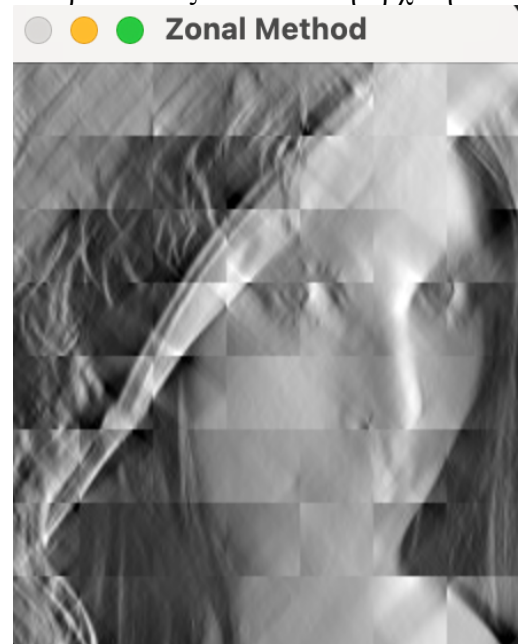
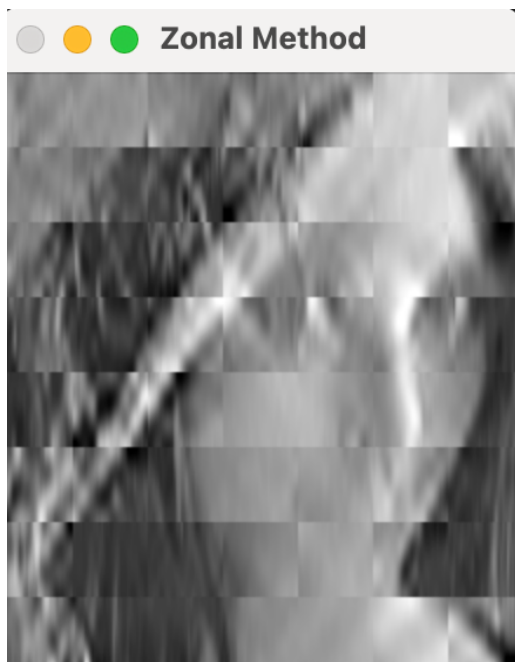
```
Οι συντελεστές του block 35 είναι:  
[[ 3.84493726e+03 -6.31016479e+02 -1.60651243e+00 ... -1.23279448e+01  
-6.36780930e+00 -7.15752649e+00]  
[-8.70562012e+02 6.42520676e+01 5.20593933e+02 ... 3.22766352e+00  
-2.04503870e+00 5.04222488e+00]  
[-2.70722656e+02 3.16132721e+02 -3.07888092e+02 ... -8.69572735e+00  
-1.25279875e+01 1.91545773e+00]  
...  
[-1.68000546e+01 1.21886950e+01 -1.37902832e+00 ... -3.71788764e+00  
-2.32857370e+00 8.96833992e+00]  
[-1.65131512e+01 1.09018555e+01 4.02231407e+00 ... -2.48151565e+00  
-6.86817598e+00 -4.91660833e-01]  
[-9.70354843e+00 5.12131214e+00 1.91653633e+00 ... 9.78612232e+00  
-2.31695032e+00 8.98579884e+00]]  
Οι συντελεστές του block 36 είναι:  
[[ 2.7014685e+03 7.2437634e+02 4.2352787e+01 ... 1.2996844e+01  
1.0092688e+00 -1.3017644e+00]  
[ 5.6576416e+02 1.8733443e+02 5.0040887e+01 ... 1.0357315e+01
```



α) Έπειτα, κρατήσαμε τους πιο σημαντικούς συντελεστές του κάθε block της εικόνας, δηλαδή αυτούς που βρίσκονται πάνω αριστερά στην εικόνα, οι οποίοι είναι αυτοί που περιέχουν την σημαντικότερη πληροφορία. Κρατώντας ένα συγκεκριμένο ποσοστό πληροφορίας p εφαρμόσαμε την **μέθοδο ζώνης**. Εφαρμόσαμε “μάσκα” σε κάθε block της εικόνας στο πάνω αριστερό μέρος και κάναμε ανακατασκευή με τους συντελεστές που κρατήσαμε.

Για ποσοστό $p = 0.9$ των συντελεστών που θα κρατήσω, παρατηρούμε ότι στην εικόνα υπάρχει απώλεια πληροφορίας αλλά είναι αισθητό να καταλάβει κανείς ποια είναι η αρχική εικόνα.

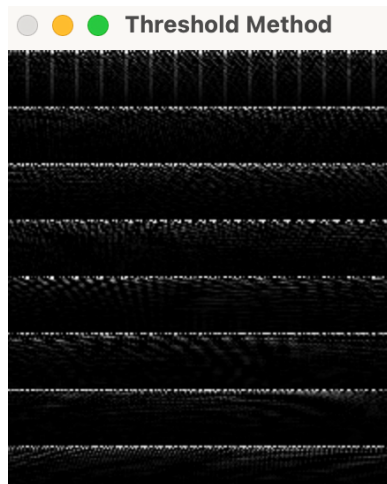
Από την άλλη, για ποσοστό $p=0.1$ στην ακριβώς από κάτω εικόνα, παρατηρείται πως χάνεται πολύ περισσότερη πληροφορία και αυτό φαίνεται από τα μάτια στο κέντρο της εικόνας.



β) Για την επιλογή των συντελεστών με την *μέθοδο κατωφλίου*, ορίσαμε την τιμή του p η οποία επηρεάζει την τιμή του κατωφλίου.

$\text{threshold} = \text{np.percentile}(\text{np.abs}(\text{merged_blocks}), 100 * (1 - p1))$

Εαν βάλουμε το $p = 1$ τότε, θα έχουμε τιμή κατωφλίου / threshold ίσο με 0, και άρα θα κρατήσει όλους τους συντελεστές. Έτσι η ανακατασκευή θα πραγματοποιηθεί με όλους τους συντελεστές, δηλαδή όλη την πληροφορία



```

Οι συντελεστές από threshold coding για p = 1:
[[ 4.5755620e+03  6.9255287e+01  2.2586528e+02 ...  9.9069965e-01
 -6.7110062e-01  4.4470006e-01]
 [ 1.5198198e+02 -3.3343056e+01 -5.1642714e+00 ... -1.5037851e+00
  2.5430691e+00  2.1590495e-01]
 [ 1.9628023e+02 -1.2459819e+01 -6.2264561e+01 ...  4.5822744e+00
 -3.6304579e+00  1.2418084e+00]
 ...
 [ 6.4367342e-01  4.4508829e+00  6.9458542e+00 ... -5.0579638e+00
 -7.6312292e-01  1.8481562e+00]
 [-5.1124573e-01 -2.5836186e+00  4.6447639e+00 ...  3.0434909e+00
 -2.9726940e-01 -1.9857417e+00]
 [-1.1892416e+00 -2.8441834e+00  6.9263892e+00 ... -1.3608630e+00
  2.4014935e-01  1.5147332e+00]]

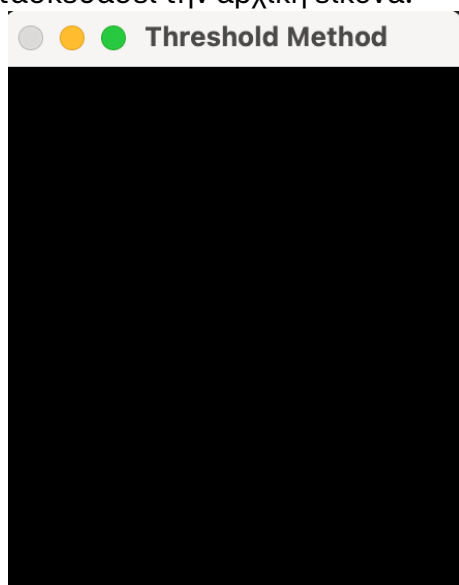
```

Από την άλλη, για $p = 0$, δηλαδή τιμή κατωφλίου 1 (100%), δεν θα κρατήσουμε κανέναν συντελεστή και άρα η ανακατασκευή θα γίνει για 0 συντελεστές, δηλαδή δεν μπορεί να ανακατασκευάσει την αρχική εικόνα.

```

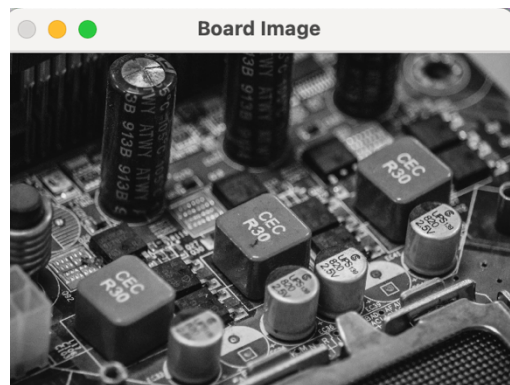
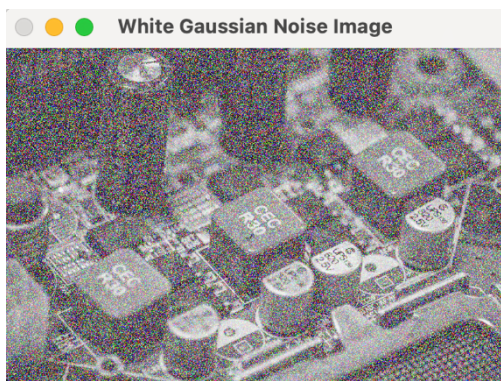
Οι συντελεστές από threshold coding για p = 0:
[[0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 ...
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]
 [0. 0. 0. ... 0. 0. 0.]]

```



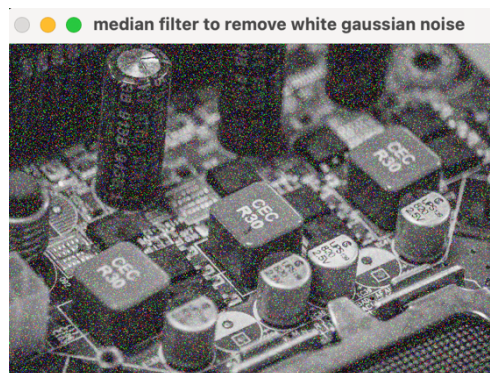
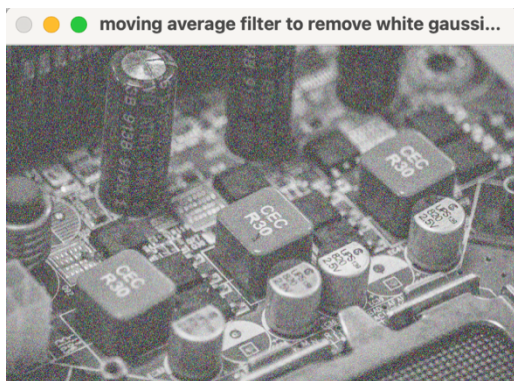
Ερώτημα 1.3 : Βελτίωση εικόνας – Φιλτράρισμα θορύβου

- 1) Προσθέσαμε στην παρακάτω εικόνα λευκό Gaussian θόρυβο με μέση τιμή ίση με το 0 και διασπορά τέτοια ώστε η τελική εικόνα να έχει λόγο σήματος προς θόρυβο ίση με 15db.

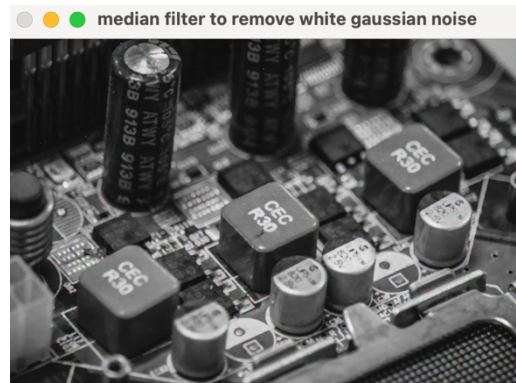
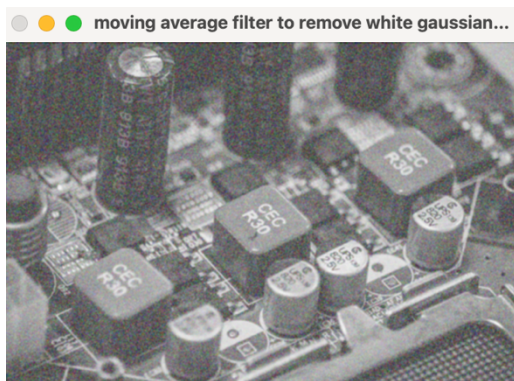


Έπειτα, ζητήθηκε να αφαιρέσουμε τον λευκό Gaussian noise με τα φίλτρα **moving average filter** και **median filter**.

Με διαστάσεις φίλτρων **3 x 3**:



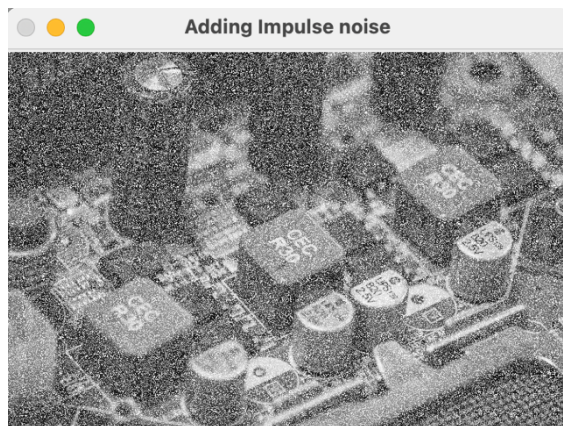
Με διαστάσεις φίλτρων **9 x 9** :



Παρατηρώ ότι **αυξάνοντας τις διαστάσεις του φίλτρου moving average filter** η αποθορυβοποίηση ελάχιστο ποσοστό του θορύβου. Αυτό το φίλτρο αφαιρεί τον θόρυβο αντικαθιστώντας κάθε pixel της οθόνης με το μέσο όρο τιμών των γειτονικών του pixel. Έτσι, αυξάνοντας τις διαστάσεις του φίλτρου τα γειτονικά pixel αυξάνονται και άρα υπάρχει περισσότερη πληροφορία θορύβου που μπορεί να αγνοηθεί. Δεν παρατηρείται κάποια ιδιαίτερη αλλαγή.

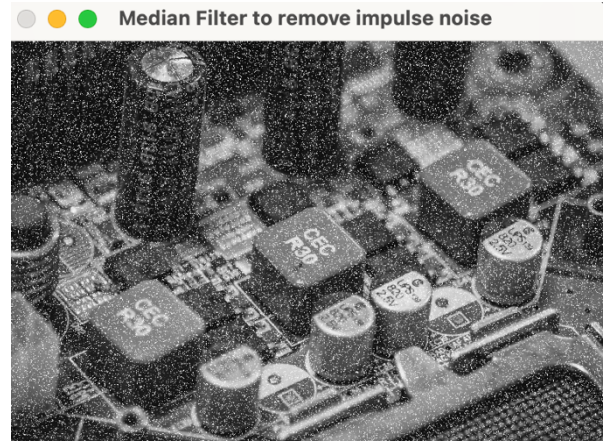
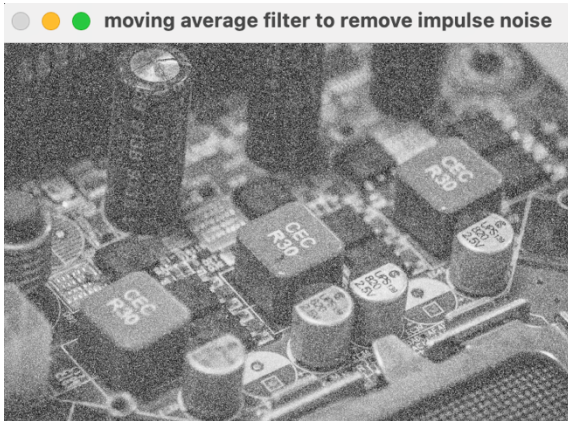
Από την άλλη, **αυξάνοντας τις διαστάσεις του φίλτρου median filter**, η αποθορυβοποίηση αφαίρεσε μεγάλο ποσοστό του θορύβου. Σε σχέση με το προηγούμενο φίλτρο, αυτό αντικαθιστά κάθε pixel με την ενδιάμεση τιμή των γειτονικών pixel, και επειδή ο θόρυβος είναι λευκός gaussian και περιέχει μεγάλες τιμές, αντικαθιστώντας με την ενδιάμεση θα αφαιρέσει αρκετό θόρυβο.

- 2) Έπειτα, προσθέσαμε τυχαία στην αρχική εικόνα κρουστικό θόρυβο με πυκνότητα / πιθανότητα 30%. Όσο μεγαλύτερη είναι η πιθανότητα τόσο πιο πολύ θα αλλοιώνεται / παραμορφώνεται η εικόνα.

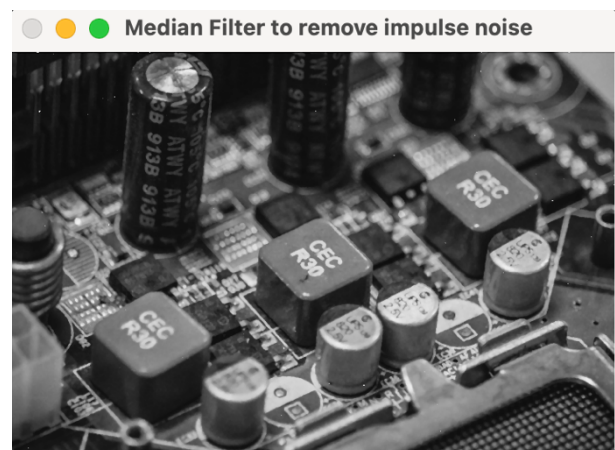
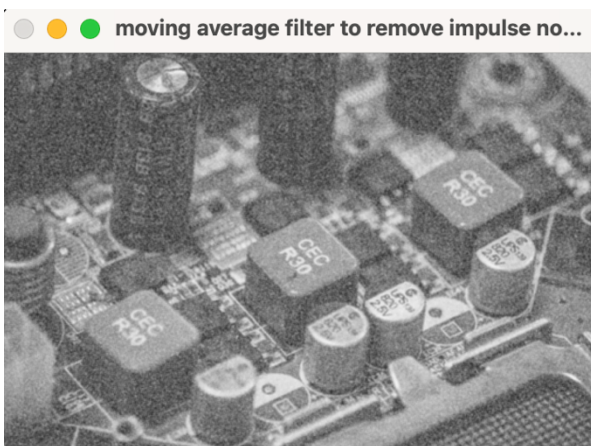


Με τα ίδια φίλτρα : moving average filters , median filter , κάναμε την αποθορυβοποίηση.

Με διαστάσεις φίλτρων **3 x 3**:



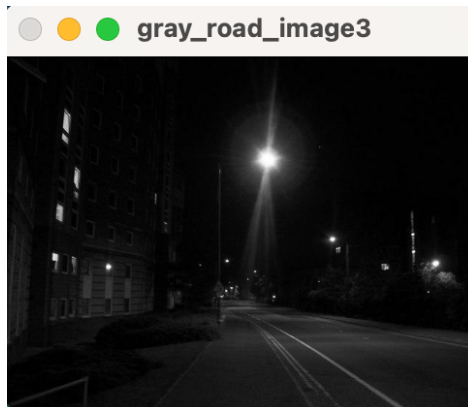
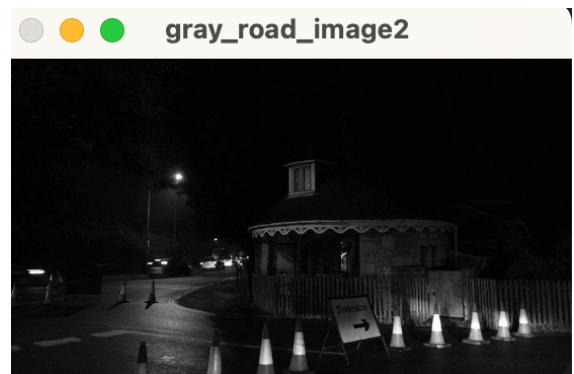
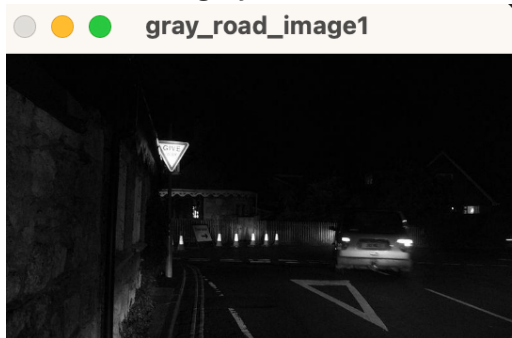
Με διαστάσεις φίλτρων **9 x 9**:



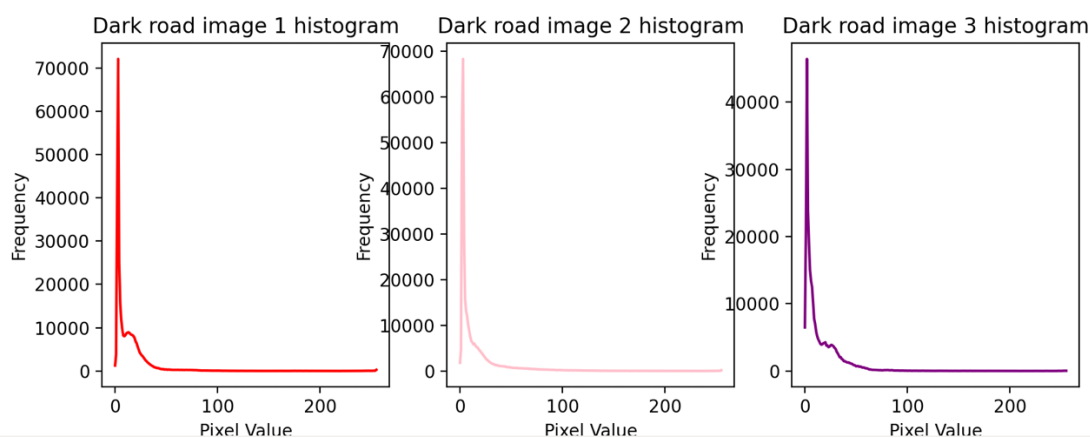
Όπως και στο προηγούμενο ερώτημα, έτσι και εδώ αυξάνοντας τις διαστάσεις του median filter, βελτιώνει σε μεγάλο βαθμό την αποθορύβοποίηση λόγω των ακραίων τιμών που έχει και αντικαθιστά τα pixel με τις ενδιάμεσες τιμές των γειτονικών.

Ερώτημα 1.4 : Βελτίωση εικόνας – Εξίσωση ιστογράμματος

- 1) Μας δόθηκαν τρεις διαφορετικές εικόνες που απεικονίζουν περιοχές σε σκοτεινές λήψεις. Αρχικά, για την εμφάνιση του histogram με grayscale τιμές, μετατρέψαμε τις εικόνες σε grayscale.



Έπειτα, εμφανίσαμε τα αντίστοιχα histogram της κάθε μία εικόνας.



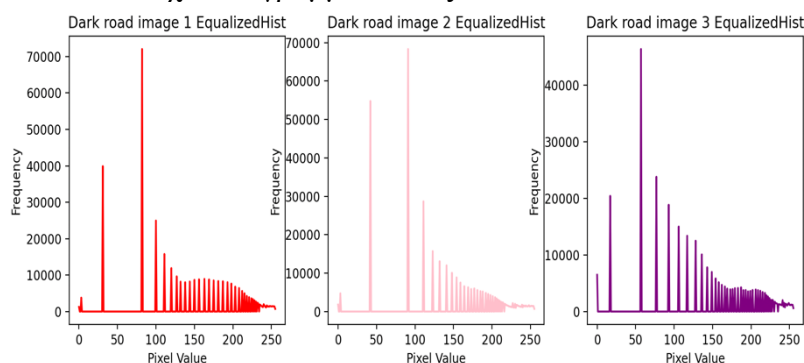
Παρατηρούμε ότι στις εικόνες υπάρχουν σημεία που αλλάζει η φωτεινότητα, από π.χ. κάποιο φως. Επειδή όμως είναι σκοτεινές εικόνες, δηλαδή οι τιμές των pixel είναι χαμηλές σχεδόν σε όλα τα σημεία, παρατηρείται το έντονο «ανέβασμα» της γραφικής κοντά στο μηδέν. Η γραφική είναι για x : τιμές των pixel και για y : συχνότητα, δηλαδή έχοντας μόνο σκούρα,

μαύρα, και γκρι χρώματα η συχνότητα για εκείνα τα pixel (κοντά στο μηδέν) θα είναι ανεβασμένη.

- 2) Με την **ολική εξίσωση ιστογράμματος** πραγματοποιείται επεξεργασία στην εικόνα τέτοια ώστε κατανεμηθεί πιο ομοιόμορφα η φωτεινότητα στην εικόνα. Οι παραπάνω εικόνες είναι πιο κοντά στο μηδέν διότι είναι σκοτεινές και με αυτή την επεξεργασία θα γίνει πιο φωτεινές, δηλαδή θα εισαχθούν και πιο “ανοιχτά” χρώματα του γκρι και έτσι το ιστόγραμμα τους θα ισορροπήσει και πέρα από το μηδέν.



Τα αντίστοιχα ιστογράμματα τους :

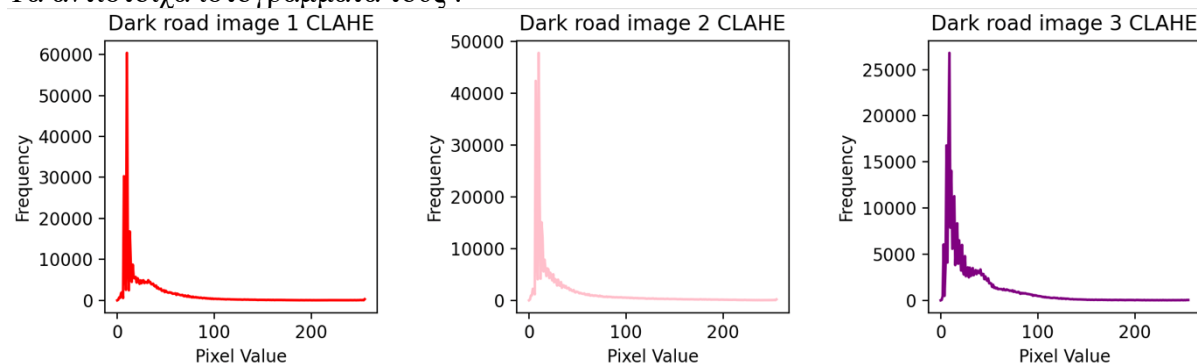


Παρατηρούνται οι αλλαγές στην φωτεινότητα των εικόνων, δηλαδή η προσθήκη και άλλων αποχρώσεων του γκρι με αποτέλεσμα το ιστόγραμμα να απλωθεί και πέρα από το 0.

- 3) Τέλος, ζητείται η εφαρμογή της **τοπικής εξίσωσης ιστογράμματος** / CLAHE, η οποία πραγματοποιεί την ίδια διαδικασία με την ολική εξίσωση αλλά πιο αποτελεσματικά λόγω το ότι μειώνει και θόρυβο.



Τα αντίστοιχα ιστογράμματα τους :



Παρατηρείται στις εικόνες μία μικρή αλλαγή στην φωτεινότητα, δηλαδή βλέπουμε και άλλες αποχρώσεις του γκρι σε σχέση με τις αρχικές εικόνες, γι' αυτό και τα ιστογράμματα τους αυξάνονται στην συχνότητα και για τιμές pixel ελαφρώς πιο μακριά από το μηδέν.

Ερώτημα 1.5 : Αποκατάσταση εικόνας (Wiener deconvolution)

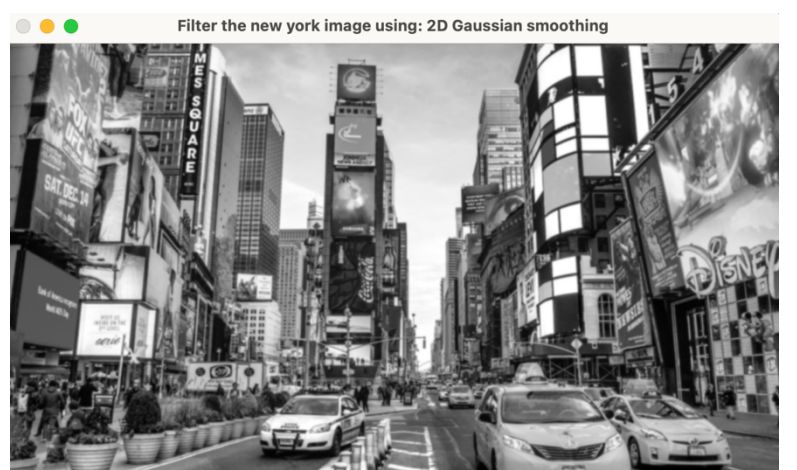
Ζητήθηκε η υποβάθμιση της δεξιάς εικόνας με χρήση του Γραμμικού Μοντέλου Υποβάθμισης.



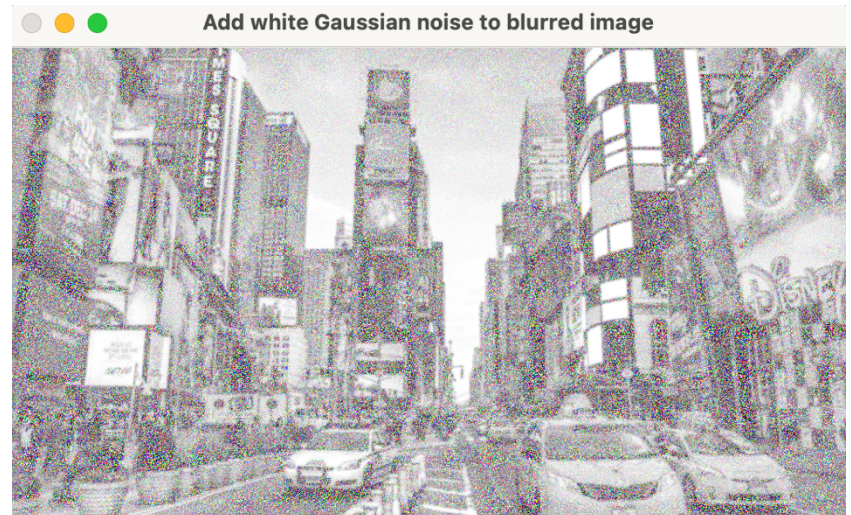
- 1) Αρχικά, φιλτράραμε με 2D πυρήνα εξομάλυνσης Gaussian, δηλαδή αφαιρέσαμε κάποιες λεπτομέρειες και θόρυβο από την εικόνα. Αυτή η διαδικασία μετατρέπει την εικόνα σε μία πιο θολή έκδοση της.
Για **τυπική απόκλιση 1.5** :



Για **τυπική απόκλιση 2.0** , παρατηρούμε ότι η εικόνα είναι πιο θολή από ότι για απόκλιση 1.5 . Άρα όσο αυξάνουμε την τυπική απόκλιση τόσο πιο ευρή κατανομή θα έχουμε και άρα πιο θολή εικόνα.



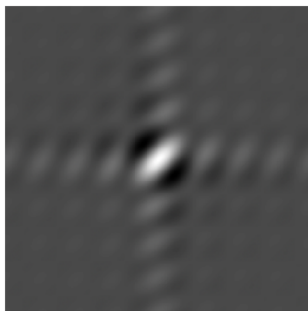
- 2) Έπειτα, προσθέσαμε στην ήδη φιλτραρισμένη εικόνα λευκό Gaussian noise με λόγο σήματος προς θόρυβο ίσο με 7db.



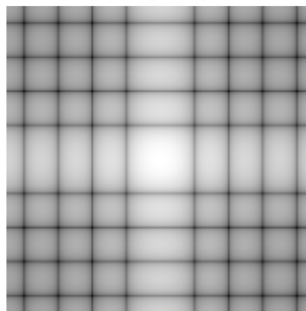
Υπολογίσαμε την **κρουστική απόκριση του φίλτρου υποβάθμισης** καθώς και την **απόκρισης συχνότητας του**. Αρχικά, δημιουργήσαμε έναν πυρήνα του φίλτρου υποβάθμισης μεγέθους 9×9 και για την κρουστική απόκριση εφαρμόσαμε τον πυρήνα του φίλτρου στην εικόνα. Έπειτα, για τον υπολογισμό της απόκρισης συχνότητας υπολογίσαμε τον μετασχηματισμό Fourier του πυρήνα του φίλτρου και από εκεί βρήκαμε την απόκριση συχνότητας. Ακόμη, για τον υπολογισμό της απόκρισης συχνότητας μετακινήσαμε τον πυρήνα του φίλτρου στο κέντρο του μετασχηματισμού έτσι ώστε οι χαμηλές συχνότητες να βρίσκονται στο κέντρο και πιο υψηλές γύρω γύρω. Για διάφορα μεγέθη πυρήνα :

Για μέγεθος πυρήνα 9×9 θα έχουμε :

fourier transf. kernel in the center

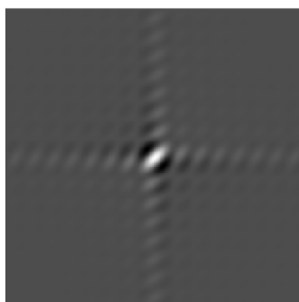


Frequency response

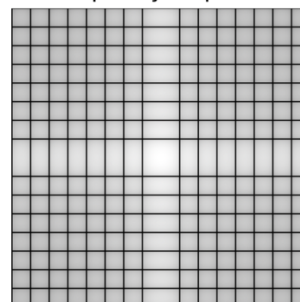


Ενώ για μέγεθος πυρήνα 16×16 θα έχουμε :

fourier transf. kernel in the center



Frequency response

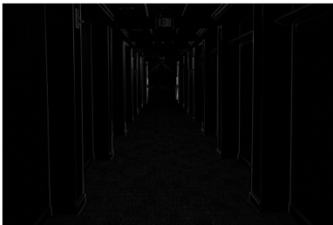


Στην συνέχεια αποκαταστήσαμε την εικόνα εφαρμόζοντας **Wiener Deconvolution**, δηλαδή απομακρύναμε τον θόρυβο. Εφόσον γνωρίζουμε την κρουστική απόκριση μπορούμε να επαναφέρουμε την αρχική εικόνα.

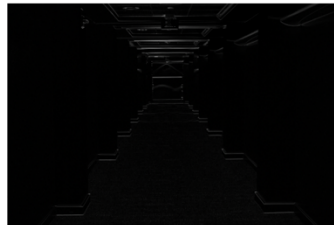
Ερώτημα 1.6 : Ανίχνευση ακμών

- 1) Πραγματοποιήθηκε εκτίμηση των συνιστωσών x , y του gradient της εικόνας με την χρήση μάσκας Sobel.

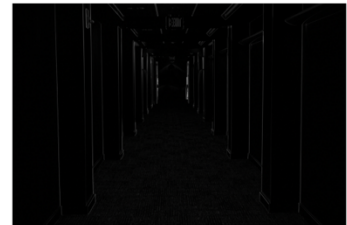
absolute sobel x



absolute sobel y

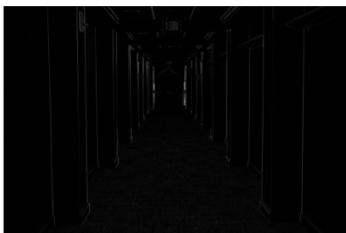


Gradient (NO gamma correction)



Με την εφαρμογή του μετασχηματισμού φωτεινότητα gamma correction γίνονται πιο ευδιάκριτη την εικόνα, γίνεται πιο φωτεινή.

absolute sobel x



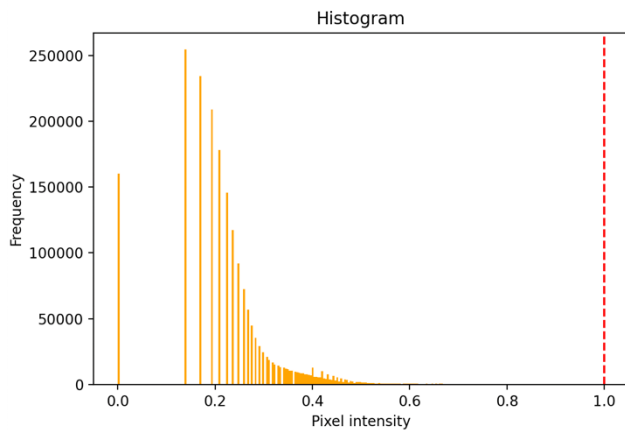
absolute sobel y



Gradient (gamma correction)



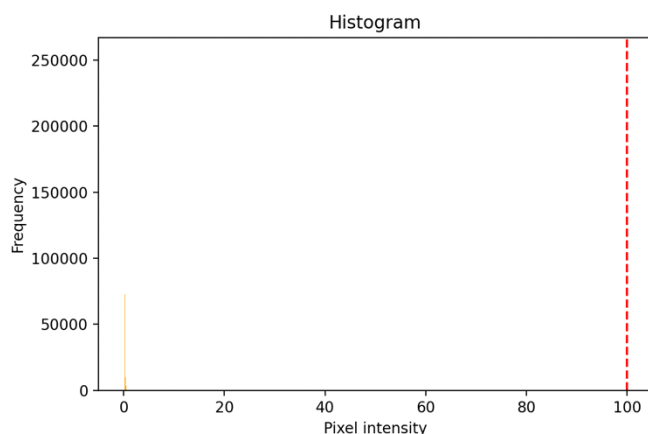
- 2) Με εφαρμογή της **ολικής κατωφλίωσης** στην εικόνα πλάτους του gradient θα γίνουν πιο διακριτές οι ακμές της.
Για τιμή κατωφλίου 1 θα έχουμε :



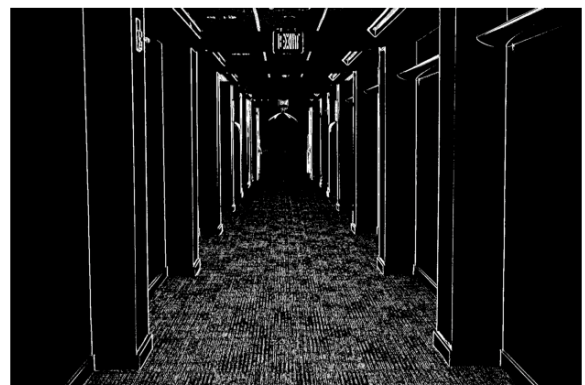
Gradient image with threshold



Για τιμή κατωφλίου 100 θα έχουμε :
Αυξάνοντας την τιμή του κατωφλίου, τόσο λιγότερες τιμές θα συμπεριληφθούν στο gradient και τόσο περισσότερες θα γίνουν μαύρες. Ακόμη, οι ακμές γίνονται πιο ευδιάκριτες αφού αυτές οι τιμές διατηρούνται στο gradient.

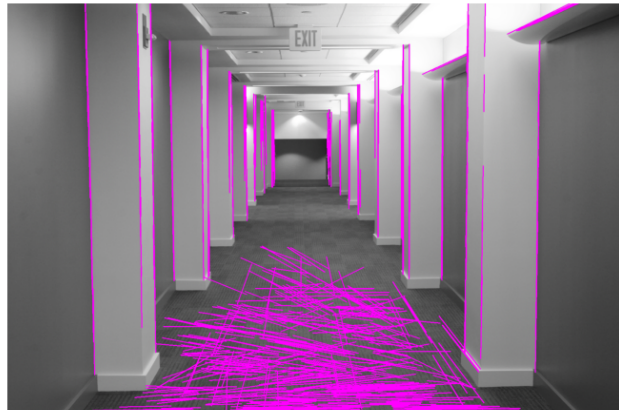


Gradient image with threshold



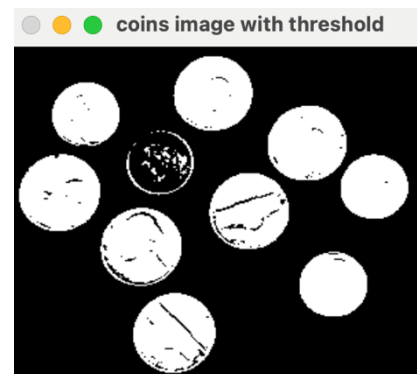
- 3) Με την εφαρμογή του μετασχηματισμού Hough έγινε ανίχνευση των ευθυγράμμων τμημάτων στην εικόνα. Για τιμή κατωφλίου 100 έχουμε τα εξής ευθύγραμμα τμήματα :
Όσο αυξάνουμε την τιμή κατωφλίου, τόσο λιγότερα τμήματα θα έχουμε αφού αγνοούνται οι τιμές που είναι μικρότερες του κατωφλίου.

Hough algorithm

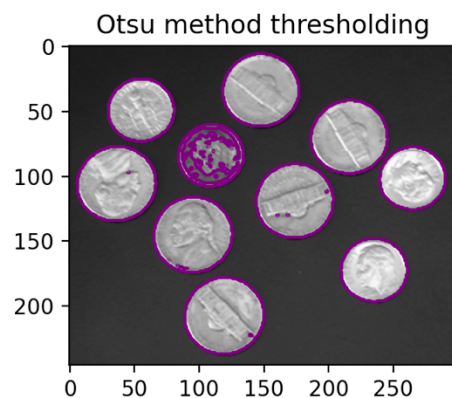
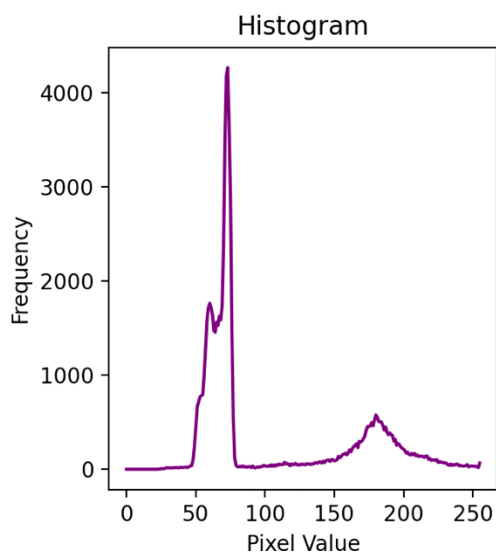


Ερώτημα 1.7 : Κατάτμηση εικόνας

- 1) Πραγματοποίηση κατάτμησης εικόνας με την ολική κατωφλίωση. `_, coins_thresholded = cv2.threshold(coins_image, threshold, 255, cv2.THRESH_BINARY)`
Για κατώφλι 150 :



- α) Έπειτα, με την μέθοδο Otsu υπολογίσαμε το ιστόγραμμα της εικόνας και το βέλτιστο κατώφλι.



- β) Η παρακάτω εικόνα περιέχει το αποτέλεσμα της κατάτμησης, όπου είναι τα κέρματα με μαύρο φόντο και χρησιμοποιήθηκε ως μάσκα τα αποτελέσματα της κατωφλίωσης. Η μάσκα “δείχνει” ποια pixels ανήκουν στο μαύρο φόντο και ποια στα κέρματα.





γ) Τέλος, παίρνουμε κάθε περιοχή που περιέχει ένα κέρμα και σε μία νέα εικόνα τα αποτυπώνουμε με διαφορετικά χρώματα και λευκό φόντο.

- 2) Με την χρήση του αλγορίθμου k-means για τιμές $k = 2, 3, 4$, δηλαδή για κάθε τιμή του k θα έχουμε αντίστοιχα τόσα clusters με αντίστοιχα χρώματα. Όσο μεγαλύτερη είναι η τιμή του k τόσο περισσότερα χρώματα θα έχουν, αφού χωρίζεται η εικόνα σε περισσότερους clusters. Για k με μικρή τιμή, τότε όλο και λιγότερα χρώματα και λεπτομέρειες θα έχει. Στην ουσία ομαδοποιεί τα pixel της αρχικής εικόνας σε διάφορες ομάδες / clusters.

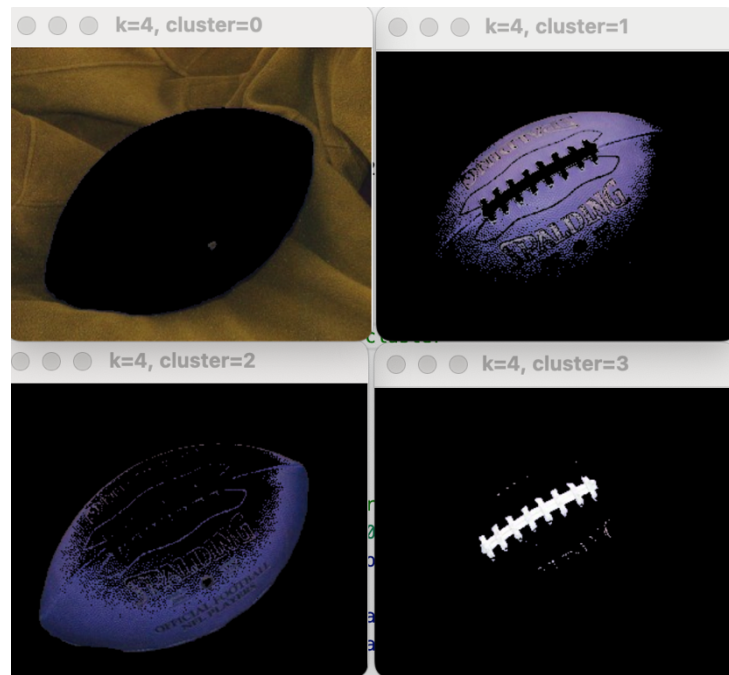
Για $k = 2$ θα έχουμε 2 clusters :



Για $k = 3$ θα έχουμε 3 clusters :



Για $k = 4$ θα έχουμε 4 clusters :



Κώδικας

E11 : Περιγραφή σχήματος

```
import cv2
import numpy as np

# διαβάζουμε την εικόνα
leaf_image = cv2.imread('leaf.jpg')

# μετατρέπουμε την εικόνα σε grayscale
grayscale_image = cv2.cvtColor(leaf_image, cv2.COLOR_BGR2GRAY)

# πως προκύπτει η δυαδική εικόνα :
# κατωφλιώνουμε με τιμή κατωφλίου τ = 220
# εάν > τ=220 --> άσπρο
# εάν < τ=220 --> μαύρο
_, binary_image = cv2.threshold(grayscale_image, 220, 255, cv2.THRESH_BINARY)

# εκτυπώνουμε τις εικόνες
# αρχική εικόνα leaf_image
cv2.imshow('leaf_image', leaf_image)
# grayscale image show
cv2.imshow('Grayscale_image', grayscale_image)
# δυαδική image show
cv2.imshow('Binary_image', binary_image)

thresholded_image = cv2.bitwise_not(binary_image) # αντιστροφή της δυαδικής
cv2.imshow('Binary_image reverse', thresholded_image)

# ΑΛΓΟΡΙΘΜΟΣ MOORE BOUNDARY TRACING
# moore boundary tracing algorithm
def moore_boundary_algorithm(image):
    boundaries, _ = cv2.findContours(image, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_NONE)
    boundary = boundaries[0] if boundaries else None
    return boundary

# εξάγουμε το περίγραμμα του φύλλου από τη δυαδική εικόνα
leaf_with_pink_boundary = moore_boundary_algorithm(thresholded_image)

color = (255, 0, 255) # χρώμα για το περίγραμμα --> ροζ
# σχεδιάζουμε το ροζ περίγραμμα πάνω στην leaf_image
cv2.drawContours(leaf_image, [leaf_with_pink_boundary], -1, color, thickness=2)
cv2.imshow('Moore boundary tracing algorithm with pink outline', leaf_image) #
εμφάνιση leaf_image με ροζ περίγραμμα
```

```

# ΠΕΡΙΓΡΑΦΕΙΣ FOURIER
# υπολογισμός --> περιγραφείς Fourier του ροζ περιγράμματος
# ροζ περίγραμμα --> σύνθετους αριθμούς
leaf_pink_boundary_to_complex = np.array(leaf_with_pink_boundary[:, 0, 0] + 1j *
leaf_with_pink_boundary[:, 0, 1], dtype=np.complex128)

# υπολογισμός : μετασχηματισμός Fourier του πίνακα με τους αριθμούς των ροζ
περιγράμματος
fourier_description = np.fft.fft(leaf_pink_boundary_to_complex)

print("Περιγραφείς Fourier του ροζ περιγράμματος:") # περιγραφείς Fourier / πίνακας
συντελεστών DFT
print(fourier_description)

# ΑΝΑΚΑΤΑΣΚΕΥΗ ΠΕΡΙΓΡΑΜΜΑΤΟΣ ΑΠΟ ΠΕΡΙΓΡΑΦΕΙΣ FOURIER (IFFT)
# σημαντικότεροι συντελεστές
# 100 : χρήση όλων των συντελεστών --> δεν χάνεται πληροφορία
# 50 --> χάνεται πληροφορία
# 10 --> χάνεται πληροφορία
# 1 --> χάνεται πληροφορία
percentages = [100, 50, 10, 1]

# IFFT
for percentage in percentages:
    #συντελεστές που θα χρησιμοποιήσω
    number_of_coefficients = int(len(fourier_description) * percentage / 100) #
    συν. αριθμός συντ. * ποσοστό / 100

    coeff_fourier_description = np.zeros_like(fourier_description) # πίνακας με
    συντ.
    #διατηρούμε τους πιο σημαντικούς συντε --> αρχή κ τέλος
    # παίρνουμε τους τελευταίους συντ --> πιο σημαντικοί και τους βάζουμε στον νέο
    πίνακα
    coeff_fourier_description[:number_of_coefficients] =
    fourier_description[:number_of_coefficients]
    coeff_fourier_description[-number_of_coefficients:] = fourier_description[-
    number_of_coefficients:]

    # IFFT
    IFFT_description_fourier = np.fft.ifft(coeff_fourier_description)

    reconstructed_image_boundary = np.array([ # ανακατασκευασμένο περίγραμμα
        np.real(IFFT_description_fourier),
        np.imag(IFFT_description_fourier)
    ]).T.reshape((-1, 1, 2))

    reconstructed_image_boundary = np.clip(reconstructed_image_boundary, 0,
    [leaf_image.shape[1], leaf_image.shape[0]])

```

```

reconstructed_image_boundary = reconstructed_image_boundary.astype(np.int32) #
--> ακέραιους

reconstructed_image = cv2.imread('leaf.jpg')

color1 = (150, 0, 150) # χρώμα για το περίγραμμα του ανακατασκευασμένου --> μωβ
cv2.drawContours(reconstructed_image, [reconstructed_image_boundary], -1,
color1, thickness=2) # μωβ περίγραμμα
cv2.imshow(f'Reconstructed image {percentage}% of most significant
coefficients', reconstructed_image)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

E11 (ερ. 5): Περιγραφή σχήματος

```

import cv2
import numpy as np

# leaf image
leaf_image = cv2.imread('leaf.jpg')

rows, columns, _ = leaf_image.shape
rotation_matrix = cv2.getRotationMatrix2D((columns/2, rows/2), 60, 1) # περιστροφή
κάτα 60 μοίρες
rotated_image = cv2.warpAffine(leaf_image, rotation_matrix, (columns, rows))

# Ολίσθηση εικόνας κατά x = 100 και y = 60
x, y = 100, 60

# Πόσο θα μετακινηθεί στον άξονα x, y
# [1, 0, x] --> μετακινεί στον x
# [0, 1, y] --> μετακινεί στον y
x_y_rotation_matrix = np.float32([[1, 0, x], [0, 1, y]])

```



```

image_rotation_matrix = cv2.warpAffine(rotated_image, x_y_rotation_matrix,
(columns, rows))

grayscale_image = cv2.cvtColor(image_rotation_matrix, cv2.COLOR_BGR2GRAY) #
grayscale
_, binary_image = cv2.threshold(grayscale_image, 220, 255, cv2.THRESH_BINARY) # τ =
220 , binary

thresholded_image = cv2.bitwise_not(binary_image) # αντιστροφή της δυαδική
cv2.imshow('Binary_image reverse', thresholded_image)

# ΑΛΓΟΡΙΘΜΟΣ MOORE BOUNDARY TRACING
# moore boundary tracing algorithm
def moore_boundary_algorithm(image):
    boundaries, _ = cv2.findContours(image, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_NONE)
    boundary = boundaries[0] if boundaries else None
    return boundary

# εξάγουμε το περίγραμμα του φύλλου από τη δυαδική εικόνα
leaf_with_pink_boundary = moore_boundary_algorithm(thresholded_image)

color = (255, 0, 255) # χρώμα για το περίγραμμα --> ροζ
# σχεδιάζουμε το ροζ περίγραμμα πάνω στην leaf_image
cv2.drawContours(leaf_image, [leaf_with_pink_boundary], -1, color, thickness=2)
cv2.imshow('Moore boundary tracing algorithm with pink outline', leaf_image) #
εμφάνιση leaf_image με ροζ περίγραμμα
cv2.namedWindow('Original Image', cv2.WINDOW_NORMAL)
cv2.namedWindow('Grayscale Image', cv2.WINDOW_NORMAL)
cv2.namedWindow('Binary Image', cv2.WINDOW_NORMAL)

cv2.imshow('Original Image', leaf_image)
cv2.imshow('Grayscale Image', grayscale_image)
cv2.imshow('Binary Image', binary_image)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

E12 : Συμπίεση εικόνας

```
import cv2
import numpy as np

# lenna.jpg
lenna_image = cv2.imread('lenna.jpg', cv2.IMREAD_GRAYSCALE)

height, width = lenna_image.shape
block_size = 32 # block διαστάσεων 32x32

dct_blocks = [cv2.dct(np.float32(lenna_image[y:y+block_size, x:x+block_size])) #
DCT για κάθε block

for y in range(0, height, block_size) for x in range(0, width, block_size) if
lenna_image[y:y+block_size, x:x+block_size].shape == (block_size, block_size)]

# για κάθε block στο dct_blocks --> εκτύπωση συντελεστών
for i, block in enumerate(dct_blocks):
    print(f"Οι συντελεστές του block {i} είναι:")
    print(block)

# block σε κάθε διάσταση
number_blocks_x = int(width / block_size)
number_blocks_y = int(height / block_size)
# όλα τα block σε μία εικόνα
rows_blocks = [np.hstack(dct_blocks[i*number_blocks_x : (i+1)*number_blocks_x ])
for i in range(number_blocks_y)] # βάζει τα block στην ίδια γραμμή
merged_blocks = np.vstack(rows_blocks) # βάζει τα block κατακόρυφα
final_blocks_image = np.where(np.isnan(merged_blocks), 0, np.uint8(merged_blocks))#
σε uint8 τύπο data, και τελική εικόνα με blocks

# ΜΕΘΟΔΟΣ ΖΩΝΗΣ
p = 0.9 # συντελεστές που θα κρατήσω (ή 0.1)

number_of_coef = int(block_size * block_size * p) # 32 * 32 * p --> συντελεστές που
θα κρατήσω σε κάθε block
the_mask = np.zeros((block_size, block_size), dtype=np.float32) # array 32*32 μάσκα

indices = np.triu_indices(block_size) # στο άνω τριγωνικό κρατάμε συντ.
the_mask[indices[0][:number_of_coef ], indices[1][:number_of_coef ]] = 1

blocks_with_coef = [block * the_mask for block in dct_blocks] # μάσκα σε κάθε block
--> block με συντ. που κρατάω

blocks_in_rows = [np.hstack(blocks_with_coef[i*number_blocks_x:
(i+1)*number_blocks_x]) for i in range(number_blocks_y)] # block x
merged_blocks1 = np.vstack(blocks_in_rows) # block y+x
```

```

idct_blocks = [cv2.idct(block) for block in blocks_with_coef] # idct

idct_blocks_in_rows = [np.hstack(idct_blocks[i*number_blocks_x:
(i+1)*number_blocks_x]) for i in range(number_blocks_y)]
merged = np.vstack(idct_blocks_in_rows)
image_zonal_coding = np.uint8(np.clip(merged, 0, 255))

# ΜΕΘΟΔΟΣ ΚΑΤΩΦΛΙΟΥ
p1 = 1 # ποσοστό p της πληροφορίας που θα κρατήσω
# κατωφλι --> για τη μέθοδο της ζώνης
# εάν συντελεστές > τιμή κατωφλίου --> κρατάμε
threshold = np.percentile(np.abs(merged_blocks), 100 * (1 - p1))
number_of_coef1 = np.where(np.abs(merged_blocks) > threshold, merged_blocks, 0)

print(f"Οι συντελεστές από threshold coding για p = {p1}:")

print(number_of_coef1)

# idct
idct_blocks_thres = [cv2.idct(block) for block in number_of_coef1]

idct_rows_thres = [np.hstack(row_blocks) for row_blocks in idct_blocks_thres] #
block σε γραμμές στήλες
merged1 = np.vstack(idct_rows_thres)
image_threshold_coding = np.uint8(np.clip(merged1, 0, 255))

# ΕΚΤΥΠΩΣΕΙΣ
cv2.imshow('lenna image', lenna_image)
cv2.imshow('2D-DCT', final_blocks_image)
cv2.imshow('Zonal Method', image_zonal_coding)
cv2.imshow('Threshold Method', image_threshold_coding)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

E13 : Βελτίωση εικόνας – Φιλτράρισμα θορύβου

```
import cv2
import numpy as np

# board_image
board_image = cv2.imread('board.png')

# ΠΡΟΣΘΗΚΗ ΛΕΥΚΟΥ GAUSSIAN ΘΟΡΥΒΟΥ
signal_board_image = np.mean(board_image ** 2) # σήμα εικόνας

variance_noise = signal_board_image / (10 ** (15 / 10)) # διασπορά θορύβου --> σήμα
/ θόρυβο = 15db

white_Gaussian_noise = np.random.normal(0, np.sqrt(variance_noise),
board_image.shape).astype(np.uint8) # λευκός Gaussian θόρυβος

noise_board_image = cv2.add(board_image, white_Gaussian_noise) # θόρυβο --> εικόνα

# φίλτρο κινουμένου μέσου (moving average filter) & φίλτρο μεσαίου (median filter)
filter_size = 3 # διαστάσεις # 3x3 ή 9x9
filter_size1 = 9
#filter_size2 = 32

moving_average_filter = cv2.blur(noise_board_image, (filter_size1, filter_size1))
median_filter = cv2.medianBlur(noise_board_image, filter_size1)

# ΕΙΣΑΓΩΓΗ ΚΡΟΥΣΤΙΚΟΥ ΘΟΡΥΒΟΥ
# κρουστικός θόρυβος με πιθανότητα : 30%
kernel_noise = np.zeros(board_image.shape[:2], dtype=np.uint8)
cv2.randu(kernel_noise, 0, 100) # με random εισαγωγή του κρουστικού , RGB
kernel_noise[kernel_noise < 30] = 255 # 30/100 ασπρά
kernel_noise[kernel_noise < 255] = 0 # υπόλοιπα μαύρα

# κρουστικός θόρυβος --> board_image
noise_mask = cv2.merge([kernel_noise] * 3) # Δημιουργία μάσκας θορύβου
noisy_board_image1 = cv2.add(board_image, noise_mask)

# φίλτρο κινουμένου μέσου (moving average filter) & φίλτρο μεσαίου (median filter)
moving_average_filter1 = cv2.blur(noisy_board_image1, (filter_size1, filter_size1))
median_filter1 = cv2.medianBlur(noisy_board_image1, filter_size1)

cv2.imshow('Board Image', board_image)
cv2.imshow('White Gaussian Noise Image', noise_board_image)
cv2.imshow('moving average filter to remove white gaussian noise',
moving_average_filter)
cv2.imshow('median filter to remove white gaussian noise', median_filter)
```

```

cv2.imshow('Adding Impulse noise', noisy_board_image1)
cv2.imshow('moving average filter to remove impulse noise', moving_average_filter1)
cv2.imshow('Median Filter to remove impulse noise', median_filter1)

cv2.waitKey(0)
cv2.destroyAllWindows()

```

E14 : Βελτίωση εικόνας – Εξίσωση ιστογράμματος

```

import cv2
import matplotlib.pyplot as plt

# board_image
road_image1 = cv2.imread('dark_road_1.jpg')
road_image2 = cv2.imread('dark_road_2.jpg')
road_image3 = cv2.imread('dark_road_3.jpg')

gray_road_image1 = cv2.cvtColor(road_image1, cv2.COLOR_BGR2GRAY)
cv2.imshow('gray_road_image1', gray_road_image1)
hist1 = cv2.calcHist([gray_road_image1], [0], None, [256], [0, 256])

gray_road_image2 = cv2.cvtColor(road_image2, cv2.COLOR_BGR2GRAY)
cv2.imshow('gray_road_image2', gray_road_image2)
hist2 = cv2.calcHist([gray_road_image2], [0], None, [256], [0, 256])

gray_road_image3 = cv2.cvtColor(road_image3, cv2.COLOR_BGR2GRAY)
cv2.imshow('gray_road_image3', gray_road_image3)
hist3 = cv2.calcHist([gray_road_image3], [0], None, [256], [0, 256])

# ΟΛΙΚΗ ΕΞΙΣΩΣΗ HISTOGRAM
equalizeHist1 = cv2.equalizeHist(gray_road_image1)
cv2.imshow('Dark road image 1 EqualizedHist', equalizeHist1)
finalHist1 = cv2.calcHist([equalizeHist1], [0], None, [256], [0, 256])

equalizeHist2 = cv2.equalizeHist(gray_road_image2)
cv2.imshow('Dark road image 2 EqualizedHist', equalizeHist2)
finalHist2 = cv2.calcHist([equalizeHist2], [0], None, [256], [0, 256])

equalizeHist3 = cv2.equalizeHist(gray_road_image3)
cv2.imshow('Dark road image 3 EqualizedHist', equalizeHist3)
finalHist3 = cv2.calcHist([equalizeHist3], [0], None, [256], [0, 256])

```



```

# ΤΟΠΙΚΗ ΕΞΙΣΩΣΗ HISTOGRAM
clahe = cv2.createCLAHE(clipLimit=2.0, tileGridSize=(8,8))

equalize_Hist1 = clahe.apply(gray_road_image1)
cv2.imshow('Dark road image 1 CLAHE', equalize_Hist1)
final_Hist1 = cv2.calcHist([equalize_Hist1], [0], None, [256], [0, 256])

equalize_Hist2 = clahe.apply(gray_road_image2)
cv2.imshow('Dark road image 2 CLAHE', equalize_Hist2)
final_Hist2 = cv2.calcHist([equalize_Hist2], [0], None, [256], [0, 256])

equalize_Hist3 = clahe.apply(gray_road_image3)
cv2.imshow('Dark road image 3 CLAHE', equalize_Hist3)
final_Hist3 = cv2.calcHist([equalize_Hist3], [0], None, [256], [0, 256])

```

```

#ΕΚΤΥΠΩΣΗ HISTOGRAM
plt.figure(figsize=(8, 4))

```

```

plt.subplot(1, 3, 1)
plt.plot(hist1, color='red')
plt.title('Dark road image 1 histogram')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

```

```

plt.subplot(1, 3, 2)
plt.plot(hist2, color='pink')
plt.title('Dark road image 2 histogram')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

```

```

plt.subplot(1, 3, 3)
plt.plot(hist3, color='purple')
plt.title('Dark road image 3 histogram')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

```

```

# ΕΚΤΥΠΩΣΗ ΤΩΝ ΟΛΙΚΗ ΕΞΙΣΩΣΗ HISTOGRAM
plt.figure(figsize=(8, 4))

```

```

plt.subplot(1, 3, 1)
plt.plot(finalHist1, color='red')
plt.title('Dark road image 1 EqualizedHist')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

```

```

plt.subplot(1, 3, 2)
plt.plot(finalHist2, color='pink')

```

```

plt.title('Dark road image 2 EqualizedHist')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

plt.subplot(1, 3, 3)
plt.plot(finalHist3, color='purple')
plt.title('Dark road image 3 EqualizedHist')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

#ΕΚΤΥΠΩΣΗ ΤΩΝ ΤΟΠΙΚΗ ΕΞΙΣΩΣΗ HISTOGRAM
plt.figure(figsize=(8, 4))
plt.subplot(1, 3, 1)
plt.plot(final_Hist1, color='red')
plt.title('Dark road image 1 CLAHE')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

plt.subplot(1, 3, 2)
plt.plot(final_Hist2, color='pink')
plt.title('Dark road image 2 CLAHE')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

plt.subplot(1, 3, 3)
plt.plot(final_Hist3, color='purple')
plt.title('Dark road image 3 CLAHE')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
cv2.waitKey(0)
cv2.destroyAllWindows()

```

E15 : Αποκατάσταση εικόνας – (Wiener Deconvolution)

```
import cv2
import numpy as np
from matplotlib import pyplot as plt
from scipy.signal import wiener

# new_york_image
newYork_image = cv2.imread('new_york.png')
cv2.imshow('New York image', newYork_image)

# BGR --> RGB
newYork_image_rgb = cv2.cvtColor(newYork_image, cv2.COLOR_BGR2RGB)
standard_deviation = 2.0 # τυπική απόκλιση από [1.5, 2.0]
# 2D πυρήνα εξομάλυνσης Gauss filter
final_blurred_image = cv2.GaussianBlur(newYork_image_rgb, (0, 0),
standard_deviation)

# ΠΡΟΣΘΗΚΗ ΛΕΥΚΟΥ ΘΟΡΥΒΟΥ GAUSSIAN
power = np.mean(final_blurred_image ** 2) # ενέργεια σήματος
variance_noise = power / (10 ** (7 / 10)) # διασπορά θορύβου
white_Gaussian_noise = np.random.normal(0, np.sqrt(variance_noise),
final_blurred_image.shape).astype(np.uint8)
noisy_image = cv2.add(final_blurred_image, white_Gaussian_noise) # white gaussian
noise add --> final_blurred_image
print("Shape of noisy_image:", noisy_image.shape)

# οι τιμές των pixel --> εύρος [0, 255]
noisy_image = np.clip(noisy_image, 0, 255).astype(np.uint8)

# ΚΡΟΥΣΤΙΚΗ ΑΠΟΚΡΙΣΗ & ΑΠΟΚΡΙΣΗ ΣΥΧΝΟΤΗΤΑΣ ΤΟΥ ΦΙΛΤΡΟΥ
ΥΠΟΒΑΘΜΙΣΗΣ
kernel_size = 9 # πυρήνα φίλτρου υποβάθμισης (9 x 9)
kernel = np.ones((kernel_size, kernel_size), np.float32) / (kernel_size *
kernel_size)
filtered_image = cv2.filter2D(final_blurred_image, -1, kernel)

print("Shape of kernel:", np.ones([kernel_size, kernel_size]).shape)

fourier_kernel = np.fft.fft2(kernel, s=(256, 256)) # κρουστική απόκριση --> #
fourier στον πυρήνα
fourier_shift_kernel = np.fft.fftshift(fourier_kernel) # μετατόπιση του μετασχ.
πυρήνα στο κέντρο
Frequency_response_kernel = 20 * np.log(np.abs(fourier_shift_kernel) + 1e-10) #
απόκριση συχνότητας
```

```

# WIENER DECONVOLUTION
gray_noisy_image = cv2.cvtColor(noisy_image, cv2.COLOR_RGB2GRAY)
estimated_image = wiener(gray_noisy_image, mysize=[kernel_size, kernel_size],
noise=variance_noise)

cv2.imshow('New York image', newYork_image_rgb)
cv2.imshow('Filter the new york image using: 2D Gaussian smoothing',
final_blurred_image)
cv2.imshow('Add white Gaussian noise to blurred image', noisy_image)
cv2.imshow('Wiener Deconvolution Result', estimated_image)

plt.figure(figsize=(8, 4))

plt.subplot(122)
plt.imshow(Frequency_response_kernel, cmap='gray')
plt.title('Frequency response')
plt.axis('off')

plt.subplot(121)
plt.imshow(np.real(fourier_shift_kernel), cmap='gray', interpolation='none')
plt.title('fourier transf. kernel in the center')
plt.axis('off')

plt.show()

cv2.waitKey(0)
cv2.destroyAllWindows()

```

E16 : Ανίχνευση ακμών

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# hallway image , γκρι
hallway = cv2.imread('hallway.png')
hallway_image = cv2.imread('hallway.png', cv2.IMREAD_GRAYSCALE)

x_sobel = cv2.Sobel(hallway_image, cv2.CV_64F, 1, 0, ksize=3) # sobel στον x
absolute_x_sobel = np.abs(x_sobel) # απόλυτη τιμή συνιστώσας

y_sobel = cv2.Sobel(hallway_image, cv2.CV_64F, 0, 1, ksize=3) # sobel στον y
absolute_y_sobel = np.abs(y_sobel)

image_gradient = np.sqrt(x_sobel**2 + y_sobel**2) # gradient

# ΜΕΤΑΣΧΗΜΑΤΙΣΜΟΣ ΦΩΤΕΙΝΟΤΗΤΑΣ --> GAMMA CORRECTION
def gamma_correction(image, gamma):
    return np.power(image / np.max(image), gamma)

gamma = 0.3
#absolute_x_sobel_gamma = gamma_correction(absolute_x_sobel, gamma)
#absolute_y_sobel_gamma = gamma_correction(absolute_y_sobel, gamma)
image_gradient_gamma = gamma_correction(image_gradient, gamma)

# ΟΛΙΚΗ ΚΑΤΩΦΛΙΩΣΗ ΣΤΗΝ ΕΙΚΟΝΑ ΠΛΑΤΟΥΣ ΤΟΥ GRADIENT
threshold = 100 # τιμή κατωφλίου
_, thresholded_image = cv2.threshold(image_gradient_gamma, threshold / 255, 1,
cv2.THRESH_BINARY)

thresholded_image = thresholded_image.astype(np.uint8)

# ΜΕΤΑΣΧΗΜΑΤΙΣΜΟΣ HOUGH
Hough_lines = cv2.HoughLinesP(
    thresholded_image, rho=1, theta=np.pi/180, threshold=50, minLineLength=100,
    maxLineGap=10
)

if Hough_lines is not None:
    for line in Hough_lines:
        x1, y1, x2, y2 = line[0]
```



```

        cv2.line(hallway, (x1, y1), (x2, y2), (255, 0, 255), 2)
    else:
        print("No lines.")

fig, axs = plt.subplots(1, 3, figsize=(24, 6))

axs[0].imshow(absolute_x_sobel, cmap='gray')
axs[0].set_title('absolute sobel x')
axs[0].axis('off')

axs[1].imshow(absolute_y_sobel, cmap='gray')
axs[1].set_title('absolute sobel y')
axs[1].axis('off')

#axs[2].imshow(image_gradient, cmap='gray')
#axs[2].set_title('Gradient (NO gamma correction)')
axs[2].imshow(image_gradient_gamma, cmap='gray')
axs[2].set_title('Gradient (gamma correction)')
axs[2].axis('off')

plt.figure()

plt.hist(image_gradient_gamma.ravel(), bins=256, range=[0, 1], color='orange')
plt.axvline(x=threshold, color='red', linestyle='--')
plt.title('Histogram')
plt.xlabel('Pixel intensity')
plt.ylabel('Frequency')

plt.figure(figsize=(6, 6))
plt.imshow(thresholded_image, cmap='gray')
plt.title('Gradient image with threshold')
plt.axis('off')

plt.figure(figsize=(8, 4))
plt.imshow(hallway, cmap='gray')
plt.title('Hough algorithm')
plt.axis('off')

plt.show()

```

E17 (ερ. 1): Κατάτμηση εικόνας

```
import cv2
import numpy as np
import matplotlib.pyplot as plt

# coins image
coins_image = cv2.imread('coins.png', cv2.IMREAD_GRAYSCALE)

threshold = 150
_, coins_thresholded = cv2.threshold(coins_image, threshold, 255,
cv2.THRESH_BINARY)

# OTSU METHOD
histogram = cv2.calcHist([coins_image], [0], None, [256], [0, 256])
_, otsu_threshold = cv2.threshold(coins_image, 0, 255, cv2.THRESH_BINARY +
cv2.THRESH_OTSU)

# ΜΑΥΡΟ ΦΟΝΤΟ , ΑΠΟΤΕΛΕΣΜΑ ΔΥΑΔΙΚΗΣ ΚΑΤΩΦΛΙΩΣΗ ΩΣ ΜΑΣΚΑ
coins_black_background = cv2.bitwise_and(coins_image, coins_image,
mask=otsu_threshold)
cv2.imshow('Black background with mask from otsu method:', coins_black_background)

coins_contours, _ = cv2.findContours(otsu_threshold, cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE) # περίγραμμα κερμάτων
coins_white_background = np.ones((coins_image.shape[0], coins_image.shape[1], 3),
dtype=np.uint8) * 255

# ΧΡΩΜΑΤΑ ΣΤΑ COINS
# χρώματα για κάθε coin --> πράσινο , μπλε, πορτοκαλί , λαδί , μπορντό , κίτρινο ,
ροζ , μωβ , κόκκινο , γκρι
coins_colors = [ (0, 255, 0), (0, 0, 255), (0, 165, 255), (128, 128, 0), (255, 0,
255), (0, 255, 255), (203, 192, 255), (128, 0, 128), (255, 0, 0), (128, 128, 128)
]

for i, contour in enumerate(coins_contours):
    color = coins_colors[i % len(coins_colors)] # κάθε κέρμα έχει διαφορετικό
χρώματα από λίστα colors
    cv2.drawContours(coins_white_background, [contour], -1, color, -1)

cv2.imshow('coins image with threshold', coins_thresholded)
cv2.imshow('coins with different colors & white background',
coins_white_background)
```

```

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.title('Histogram')
plt.plot(histogram, color='purple')
plt.xlabel('Pixel Value')
plt.ylabel('Frequency')

plt.subplot(1, 2, 2)
plt.title('Otsu method thresholding')
plt.imshow(coins_image, cmap='gray')
plt.contour(otsu_threshold, colors='purple', levels=[1])

plt.show()

cv2.waitKey(0)
cv2.destroyAllWindows()

```

E17 (ερ. 2): Κατάτμηση εικόνας

```

import numpy as np
import cv2
from sklearn.cluster import KMeans

football_image = cv2.imread('football.jpg')
football_image = cv2.cvtColor(football_image, cv2.COLOR_BGR2RGB)

# K-MEANS ALGORITHM
pixelVal = football_image.reshape((-1, 3))
k_values = [2, 3, 4]

for k in k_values:
    kmeans = KMeans(n_clusters=k) # αριθμός των cluster
    kmeans.fit(pixelVal) # pixel --> k clusters

    labels = kmeans.labels_

    for i in range(k):
        # pixels κάθε ομάδας --> football_cluster_image

```

```

cluster_indices = np.where(labels == i)[0]
football_cluster_image = np.zeros_like(football_image) # πίνακας
for pixel in cluster_indices:
    x, y = np.unravel_index(pixel, football_image.shape[:2]) # pixel --> x
    , y
    football_cluster_image[x, y] = football_image[x, y]

cv2.imshow(f'k={k}, cluster={i}', football_cluster_image)

cv2.waitKey(0)
cv2.destroyAllWindows()

```